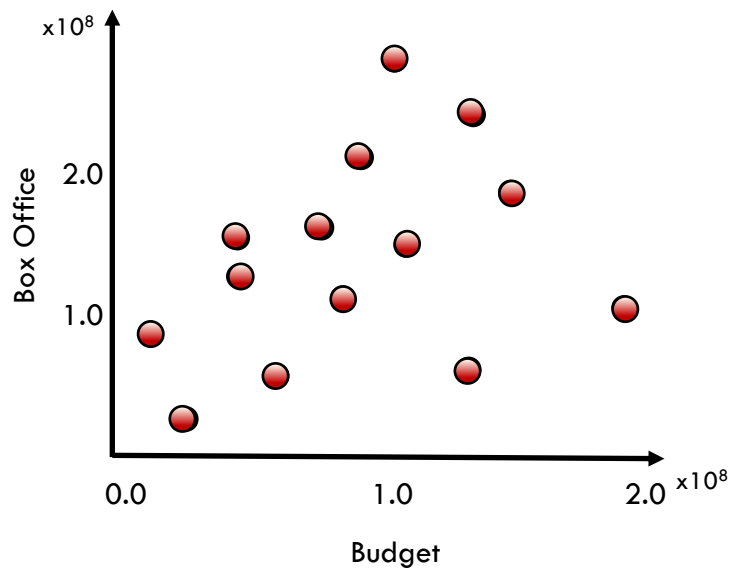


Introduction to

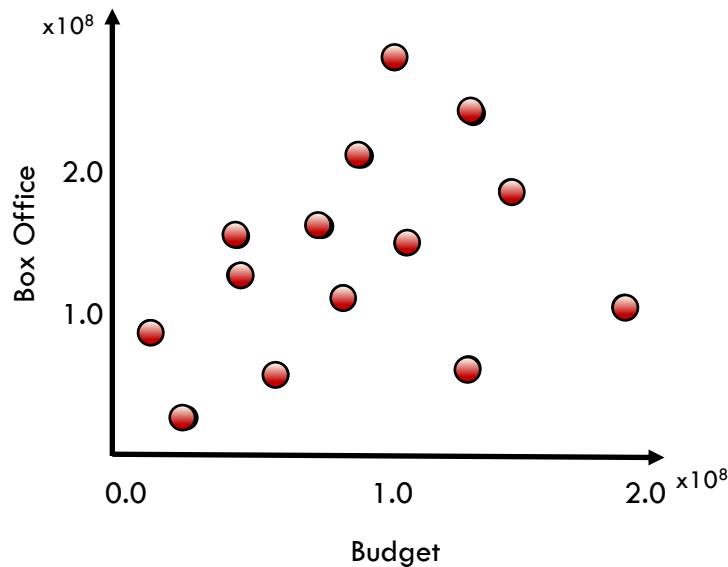
Linear Regression

Introduction to Linear Regression



$$y_{\beta}(x) = \beta_0 + \beta_1 x$$

Introduction to Linear Regression

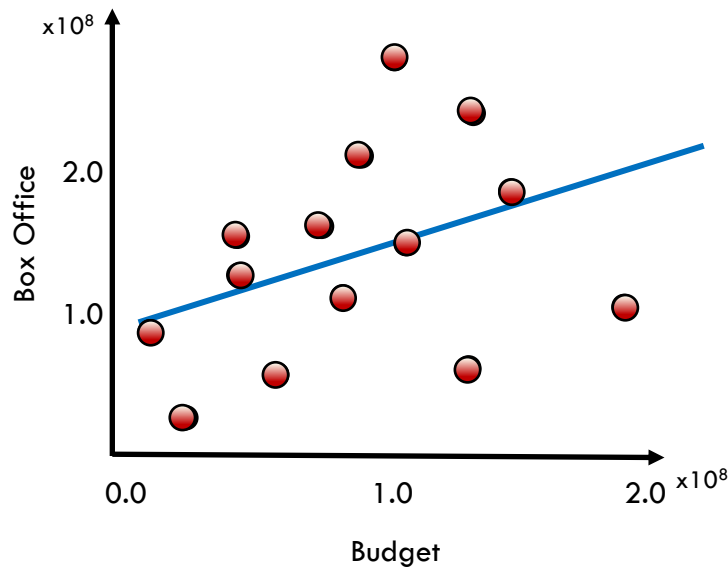


$$y_{\beta}(x) = \beta_0 + \beta_1 x$$

Diagram illustrating the linear regression equation $y_{\beta}(x) = \beta_0 + \beta_1 x$ with annotations:

- box office revenue** points to $y_{\beta}(x)$
- coefficient 0** points to β_0
- coefficient 1** points to β_1
- movie budget** points to x

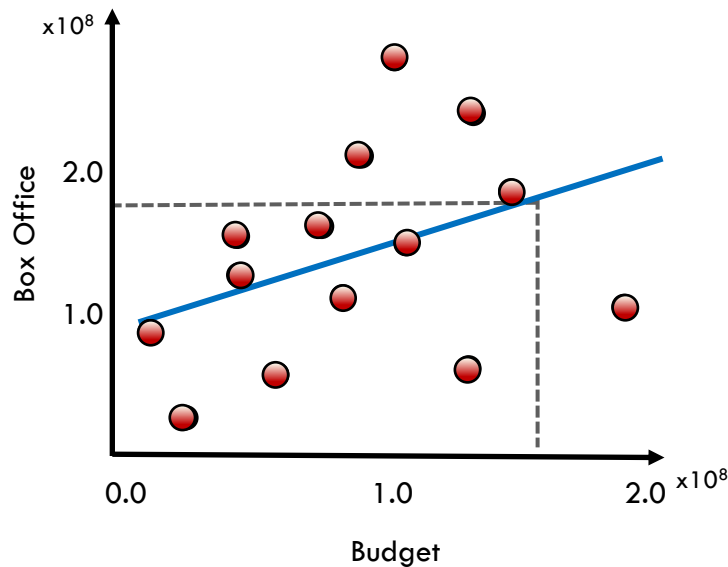
Introduction to Linear Regression



$$y_{\beta}(x) = \beta_0 + \beta_1 x$$

$$\beta_0 = 80 \text{ million}, \beta_1 = 0.6$$

Predicting from Linear Regression

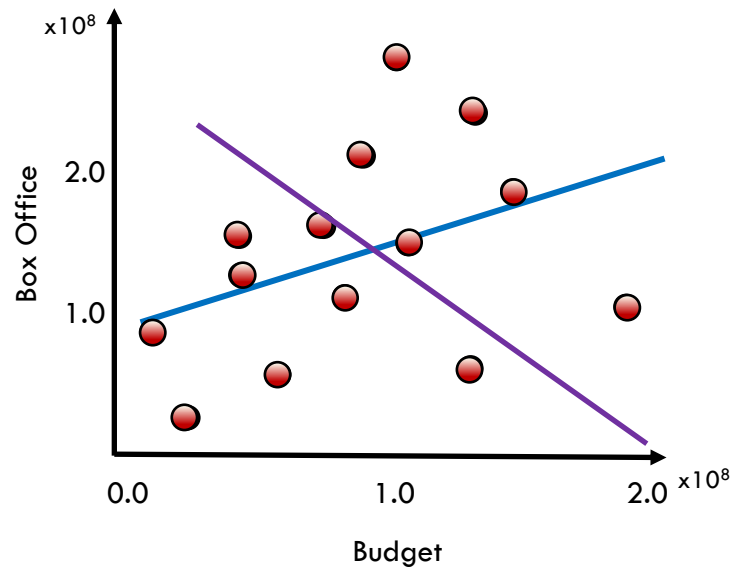


$$y_{\beta}(x) = \beta_0 + \beta_1 x$$

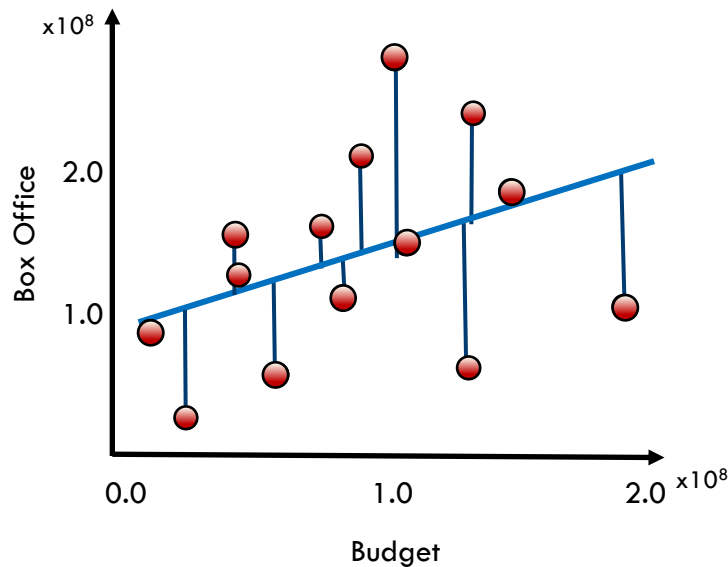
$$\beta_0 = 80 \text{ million}, \beta_1 = 0.6$$

Predict 175 Million Gross for 160 Million Budget

Which Model Fits the Best?

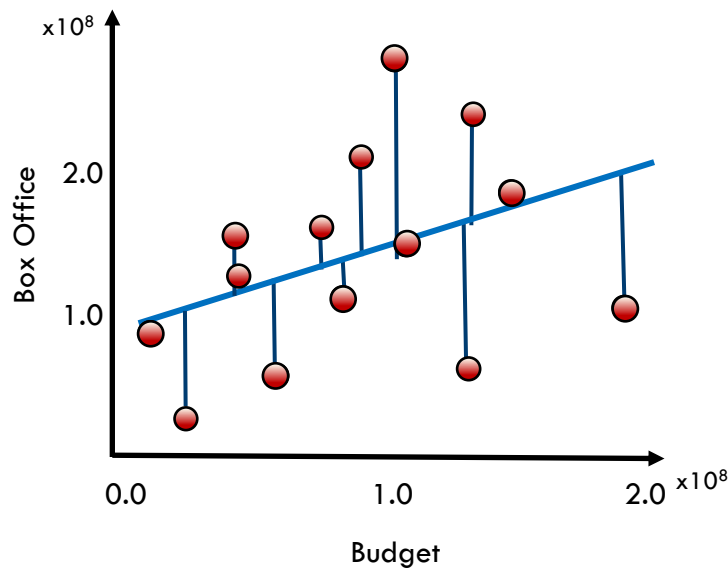


Calculating the Residuals



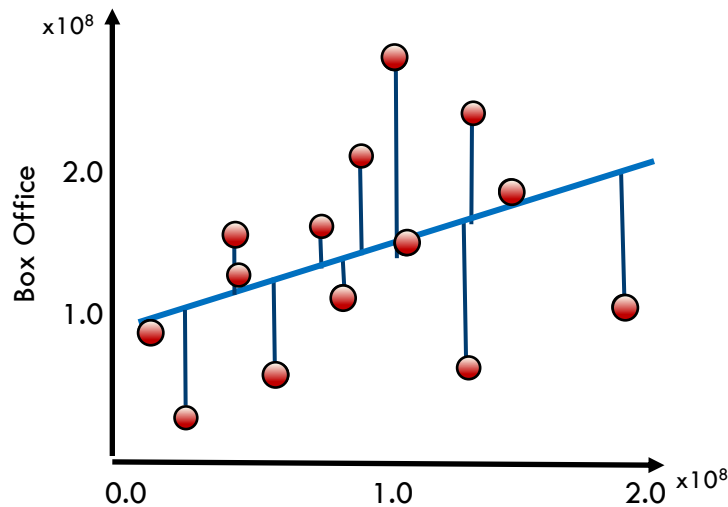
predicted value $\rightarrow y_{\beta} \left(x_{obs}^{(i)} \right) - y_{obs}^{(i)}$ $\leftarrow$ **observed value**

Calculating the Residuals



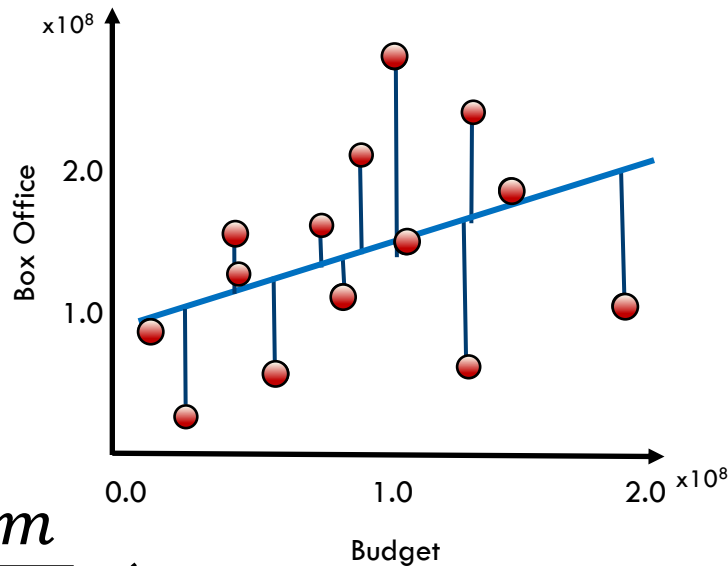
$$\left(\beta_0 + \beta_1 x_{obs}^{(i)} \right) - y_{obs}^{(i)}$$

Mean Squared Error



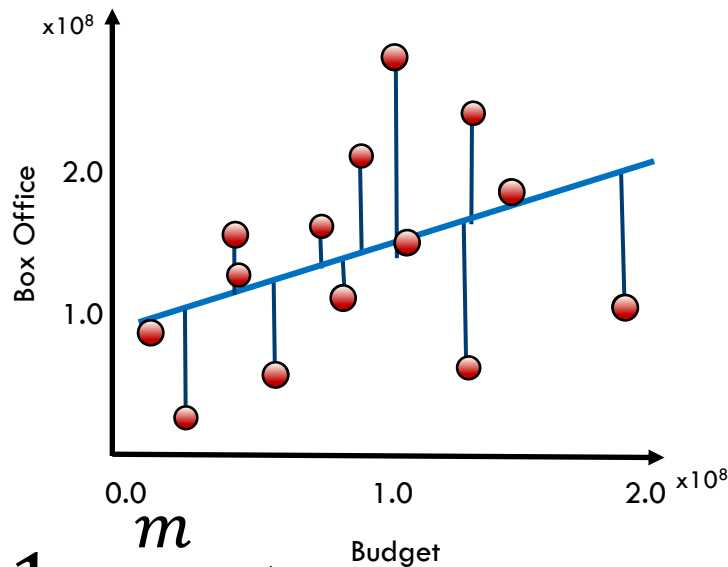
$$\frac{1}{m} \sum_{i=1}^m \left(\left(\beta_0 + \beta_1 x_{obs}^{(i)} \right) - y_{obs}^{(i)} \right)^2$$

Minimum Mean Squared Error



$$\min_{\beta_0, \beta_1} \frac{1}{m} \sum_{i=1}^m \left(\left(\beta_0 + \beta_1 x_{obs}^{(i)} \right) - y_{obs}^{(i)} \right)^2$$

Cost Function



$$J(\beta_0, \beta_1) = \frac{1}{2m} \sum_{i=1}^m \left(\left(\beta_0 + \beta_1 x_{obs}^{(i)} \right) - y_{obs}^{(i)} \right)^2$$

Modelling Best Practice

- Use cost function to fit model
- Develop multiple models
- Compare results and choose best one

Other Model Metrics

Sum of Squared Error (SSE):

$$\sum_{i=1}^m \left(y_{\beta}(x^{(i)}) - y_{obs}^{(i)} \right)^2$$

Other Measures of Error

Sum of Squared Error (SSE):
$$\sum_{i=1}^m \left(y_{\beta}(x^{(i)}) - y_{obs}^{(i)} \right)^2$$

Total Sum of Squares (TSS):
$$\sum_{i=1}^m \left(\overline{y_{obs}} - y_{obs}^{(i)} \right)^2$$

Correlation Coefficient (R^2):
$$1 - \frac{SSE}{TSS}$$

Linear Regression: The Syntax

Import the class containing the regression method

```
from sklearn.linear_model import LinearRegression
```

Create an instance of the class

```
LR = LinearRegression()
```

Fit the instance on the data and then predict the expected value

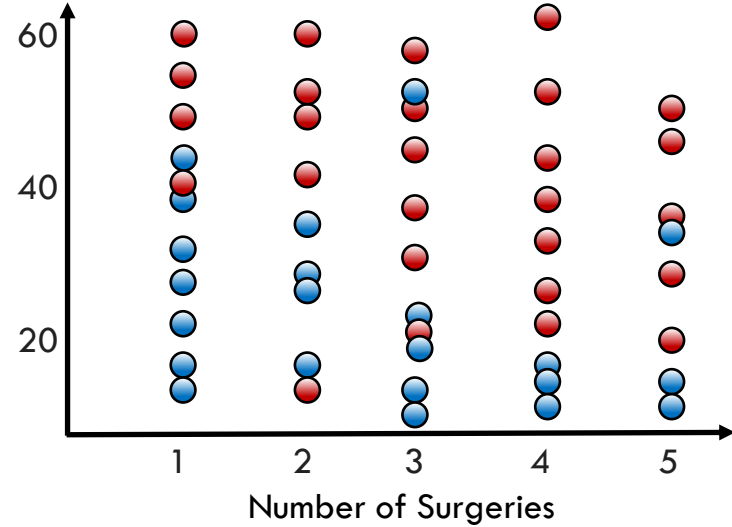
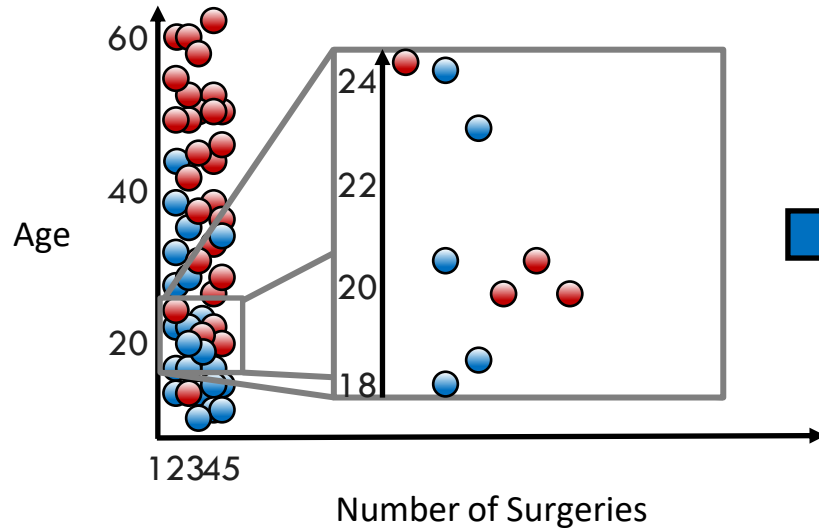
```
LR = LR.fit(X_train, y_train)
```

```
y_predict = LR.predict(X_test)
```

Advanced

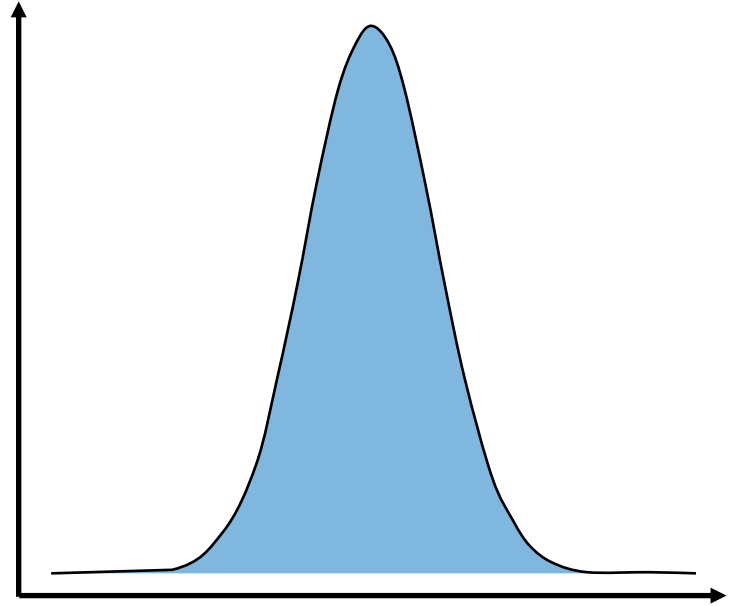
Linear Regression

Scaling is a Type of Feature Transformation



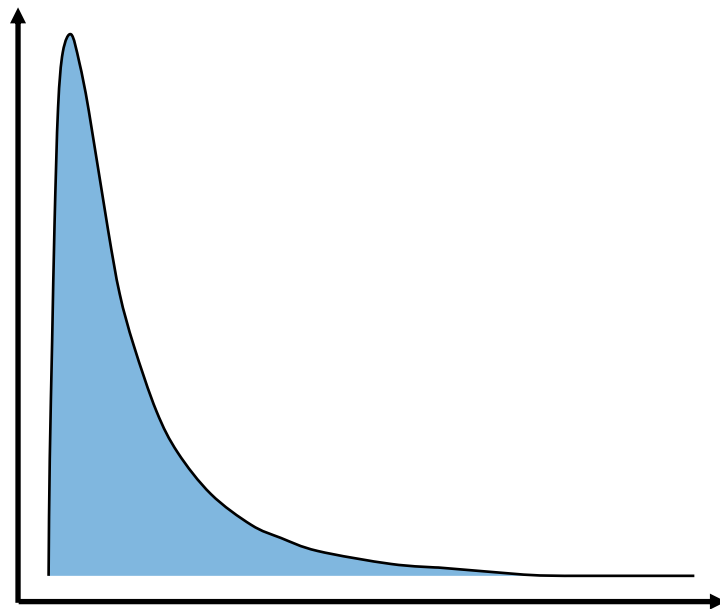
Transformation of Data Distributions

- Predictions from linear regression models assume residuals are normally distributed

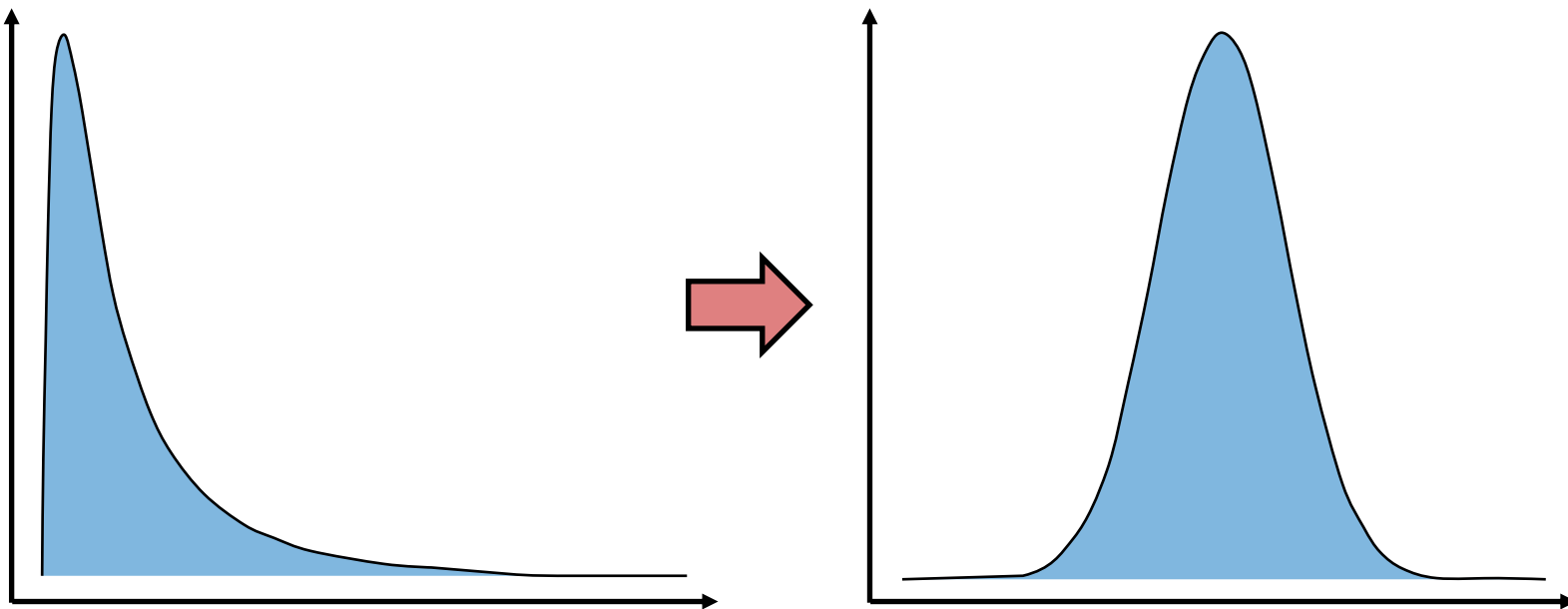


Transformation of Data Distributions

- Predictions from linear regression models assume residuals are normally distributed
- Features and predicted data are often skewed



Transformation of Data Distributions

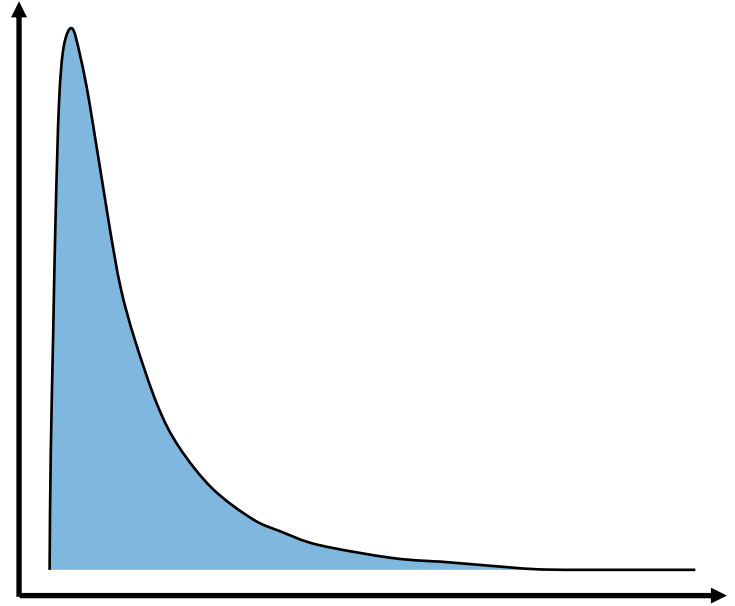


```
from numpy import log, log1p
```

```
from scipy.stats import boxcox
```

Transformation of Data Distributions

- Predictions from linear regression models assume residuals are normally distributed
- Features and predicted data are often skewed
- Data transformations can solve this issue



Feature Types

Feature Type

- **Continuous:** numerical values

Transformation

- Standard Scaling, Min-Max Scaling

Feature Types

Feature Type

- **Continuous:** numerical values
- **Nominal:** categorical, unordered features (*True* or *False*)

Transformation

- Standard Scaling, Min-Max Scaling
- One-hot encoding (0, 1)

```
from sklearn.preprocessing import LabelEncoder, LabelBinarizer, OneHotEncoder
```

Feature Types

Feature Type

- **Continuous:** numerical values
- **Nominal:** categorical, unordered features (*True* or *False*)
- **Ordinal:** categorical, ordered features (movie ratings)

Transformation

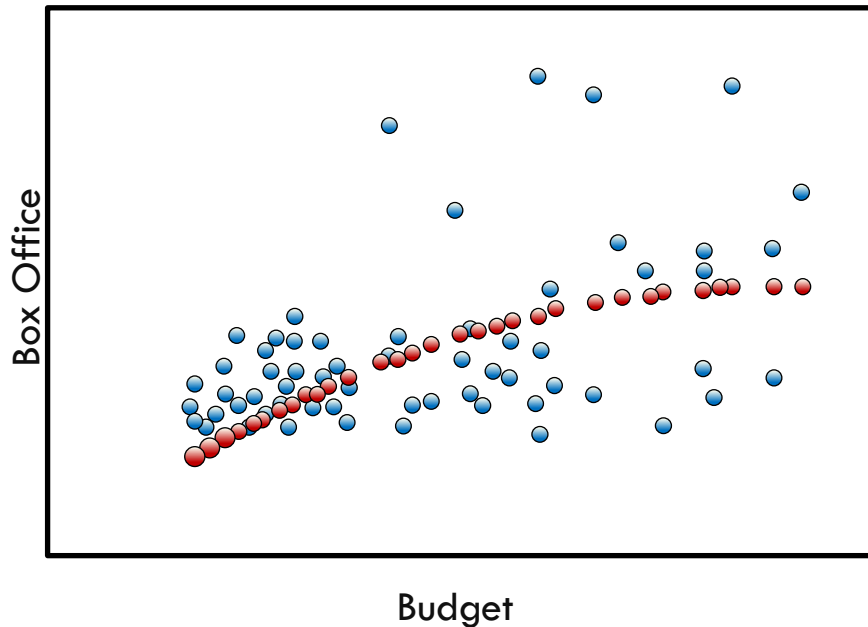
- Standard Scaling, Min-Max Scaling
- One-hot encoding (0, 1)
- Ordinal encoding (0, 1, 2, 3)

```
from sklearn.feature_extraction import DictVectorizer  
from pandas import get_dummies
```


Addition of Polynomial Features

- Capture higher order features of data by adding polynomial features

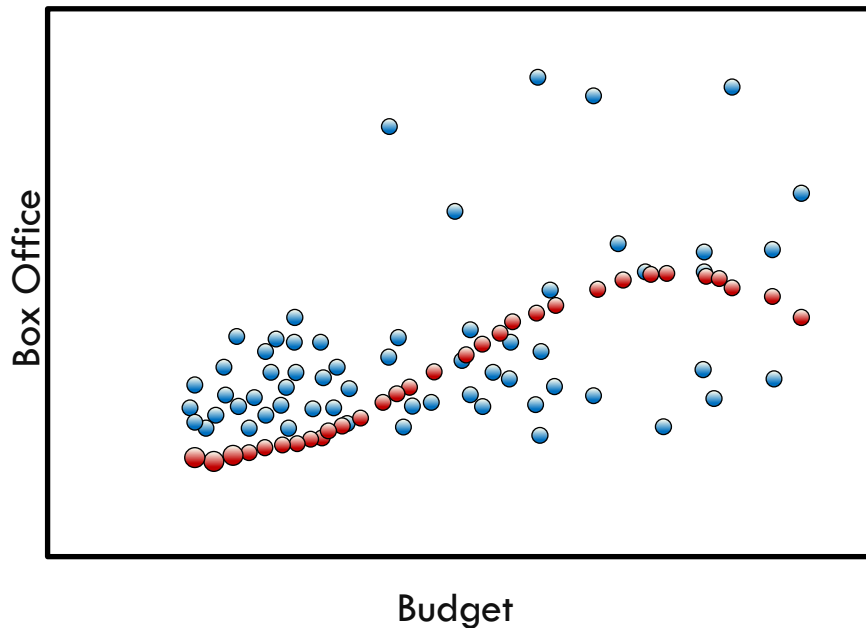
$$y_{\beta}(x) = \beta_0 + \beta_1 x + \beta_2 x^2$$



Addition of Polynomial Features

$$y_{\beta}(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3$$

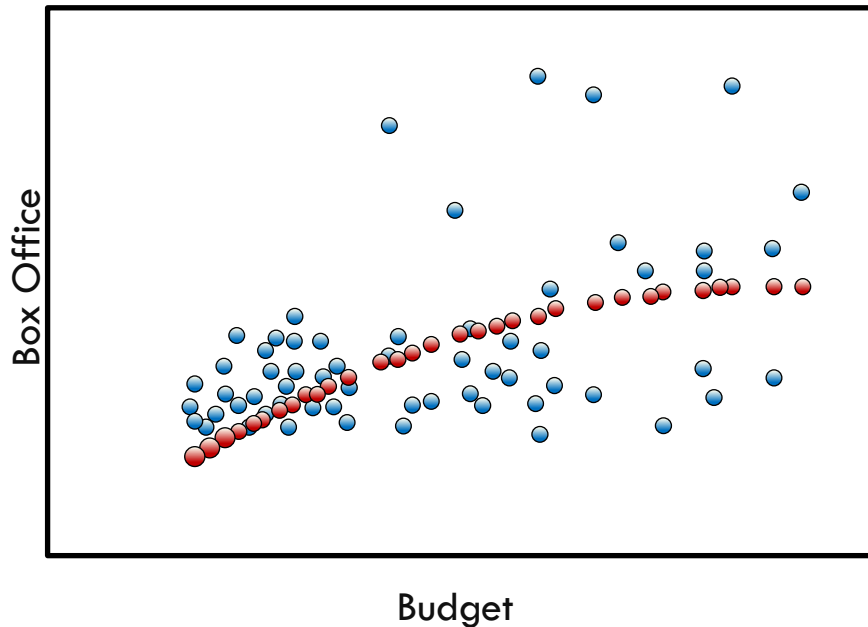
- Capture higher order features of data by adding polynomial features
- "Linear regression" means linear combinations of features



Addition of Polynomial Features

- Capture higher order features of data by adding polynomial features
- "Linear regression" means linear combinations of features

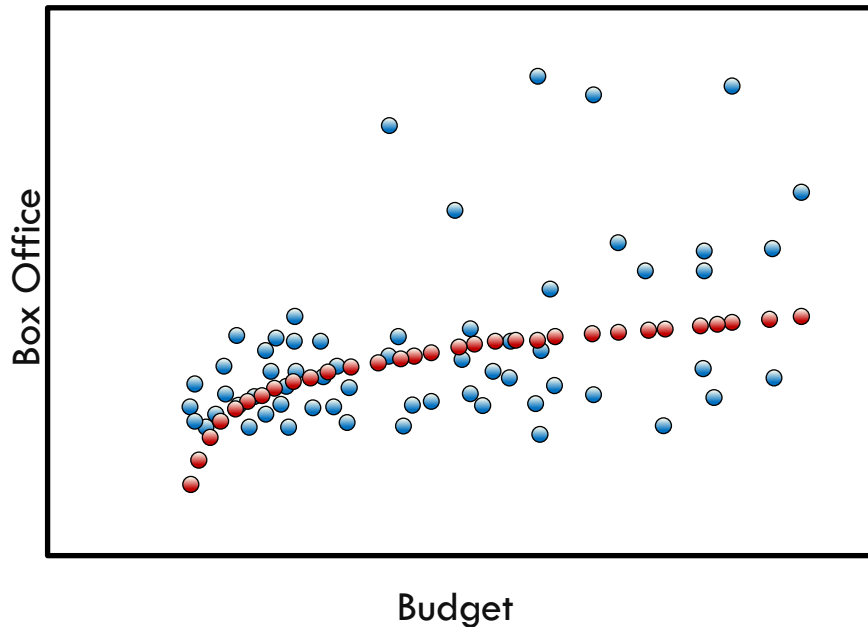
$$y_{\beta}(x) = \beta_0 + \beta_1 x + \beta_2 x^2$$



Addition of Polynomial Features

$$y_{\beta}(x) = \beta_0 + \beta_1 \log(x)$$

- Capture higher order features of data by adding polynomial features
- "Linear regression" means linear combinations of features



Addition of Polynomial Features

- Can also include variable interactions

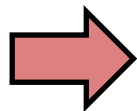
$$y_{\beta}(x) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_1 x_2$$

Addition of Polynomial Features

- Can also include variable interactions

$$y_{\beta}(x) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_1 x_2$$

- How is the correct functional form chosen?



Check relationship of each variable
or with outcome

Polynomial Features: The Syntax

Import the class containing the transformation method

```
from sklearn.preprocessing import PolynomialFeatures
```

Create an instance of the class

```
polyFeat = PolynomialFeatures(degree=2)
```

Create the polynomial features and then transform the data

```
polyFeat = polyFeat.fit(X_data)
```

```
X_poly = polyFeat.transform(X_data)
```